What a system should do when it is not sure

Daniele Del Guercio. Notes from running an LLM extraction system in production on bank NPL portfolios, and from a CE file for a motorised window. Case study: github.com/Redwae/creditiq-case-study

What I do

I take a process that today needs a person sitting there doing it by hand, and I make it run on its own.

I have done this on two grounds. One is documents and decisions: an operator who buys non-performing loan portfolios and has to price them. The other is sensors and matter: a maker of motorised window fittings, where a wrong sequence moves glass and aluminium.

What they share is not the sector. The system has to live inside someone else's process. If it is wrong, the process is wrong.

The thesis

In a CE technical file you must declare how the machine behaves in abnormal conditions. Not that it is safe. What happens if the travel sensor says the sash is open and the close limit switch is already pressed. What happens if the actuator does not answer a stop command. What happens if the limit switch never arrives and the motor would keep going. On a machine that closes a window, the safe state is stopped: cut power to the motion. Do not guess.

That discipline exists in mechanics and electronics because the standard asks for it before you mark the machine. In software projects that use a language model, nobody asks for it. The uncertain case arrives in production.

An AI project that never decided what to do when the model is not sure fails in one way, and I have seen it. The system fills the field anyway. The operator either trusts it, and a wrong decision follows, or does the work by hand again. This is not an alignment problem. You never declared the safe state.

The same discipline, on documents

The client decides whether to buy a credit, and at what price. If a mortgage filing is read as cancelled when it is not, the collateral enters the valuation cleaner than it is. The price is wrong. Downstream there is a bid, not a warning.

Documents arrive as they arrive: data tapes in Excel with different columns for each bank, PDFs, scans, forwarded emails. In Italy, and more generally in Europe, documents do not start homogeneous. Cadastre, land registry and company register do not describe the same property in the same way. Here you start from a 1996 scan and a registry formality that, if you read it wrong, changes the price.

The system refuses by field, not by document. Tax codes and VAT numbers have the highest bar: a wrong identity means you have analysed the wrong person. Next, the registered amount and the mortgage status. Address and title description: a lower bar.

Below the bar, the field does not enter the report as a fact. The case goes to an operator queue, field marked, sources next to it. The operator corrects it. The case returns from there.

I bound to a schema what has a shape: tax codes, dates, amounts, duration. I checked the rest by crossing sources. One contradiction: the cadastral record says the property was merged into a new identifier; the land registry still shows an active mortgage on the old one, and cannot find the property on the new one. The system has to notice it is looking at two histories. Not pick one.

At the start almost every new tape went to the queue. A system that refuses everything is as useless as one that is wrong. The approved mapping is kept for later files of the same shape. Refusal has to fall. Not vanish.

The same discipline, on a machine

The system had to run an open-and-close sequence: motor, limit switch, possible obstacle. Three cases I had to write down. The sensor says one thing, the limit switch another. The actuator does not move. The limit switch never arrives.

If the sensor lies, the safe state is stopped. Cut power. Do not close. CE required me to declare the three cases before powering the machine, and to write which residual risk remains acceptable. I wrote that text.

The same question applies to a software agent. When it is not sure, it must have a declared state. Otherwise the user invents the state, differently each time. On a shop floor that is not allowed. In a process where being wrong costs money, it should not be allowed on a screen either.

The loop that closes

When a person corrected the output, the correction stayed. Approved column mappings were stored with a fingerprint of the file, so the next tape from the same bank did not start from zero. Field corrections stayed on the same record, with who had corrected them.

I used those corrections to compare versions: same filing, same sheet, against what the operator had already fixed. If a new version got a field wrong that had been right, it did not go forward.

Old filings, from scans, were read worse on dates and amounts than recent system-generated ones. Without the corrections I would have seen it months later, in the prices.

The strongest signal: operators stopped cleaning the Excel sheets before uploading them. They did it when the mapping queue became faster than doing the work by hand.

What I got wrong

Evaluation should have been built first, not after. I learned that when I changed how filings were extracted and could not say whether it was better. I had to go back, take the cases already corrected, and build the comparison on them.

I had taken the model, reading the text of the filing, as the source. It is not. The parser on registry data and amounts, and the cross-check between cadastre and land registry, hold up better. The model completes. It does not decide.

A decision I reversed: let the model do everything, then validate. Now the opposite. Rules and sources first, then the model on what is left.

Something got worse. Cases piled up in the queue, and operators waited for the system instead of closing by hand.

What follows

If a company has to put agents inside a real process, three things to decide before writing a line of code. One: the safe state when the system is not sure, for each field or each action, not in general. Two: who sees a refused case, how fast it must come back, and where the correction is kept. Three: what you measure a new version against, on the same input, before it goes into production.

Most of what is written about agents assumes the next step is always another step by the agent. Inside a process where being wrong costs, the next step is often a person, a register, an authority. The agent that “goes on” is the case you never declared.

An agent with no declared safe state is not incomplete. It is a system that moves the risk onto the operator, without saying so.